

Classifying images with deep convolutional networks

Convolution · padding and stride · pooling · channels · dropout · augmentation

Running example: an AOI station on an SMT line

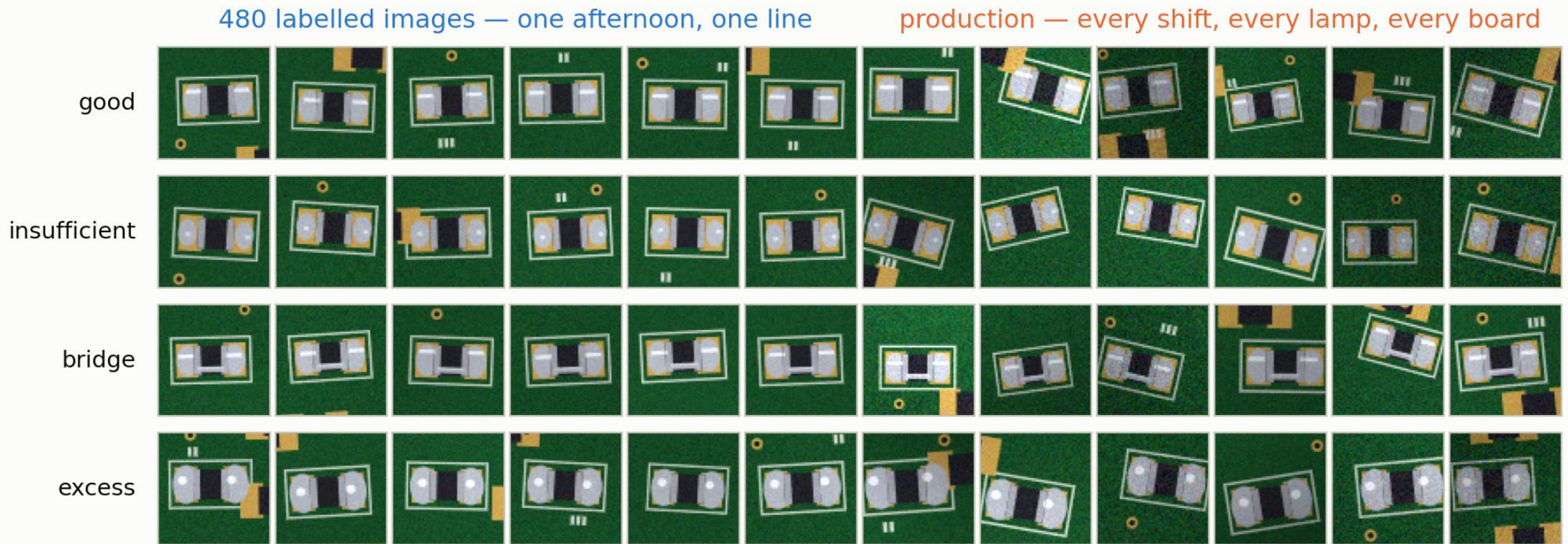
Every 0603 chip component photographed after reflow, sorted into
good · insufficient solder · bridge (a short) · excess solder

Companion notebook: [mlpi26_Lect09_nbsup1.ipynb](#)

Upper-undergraduate engineering · every number on these slides is computed in that notebook

Four classes, and a trap built into the data

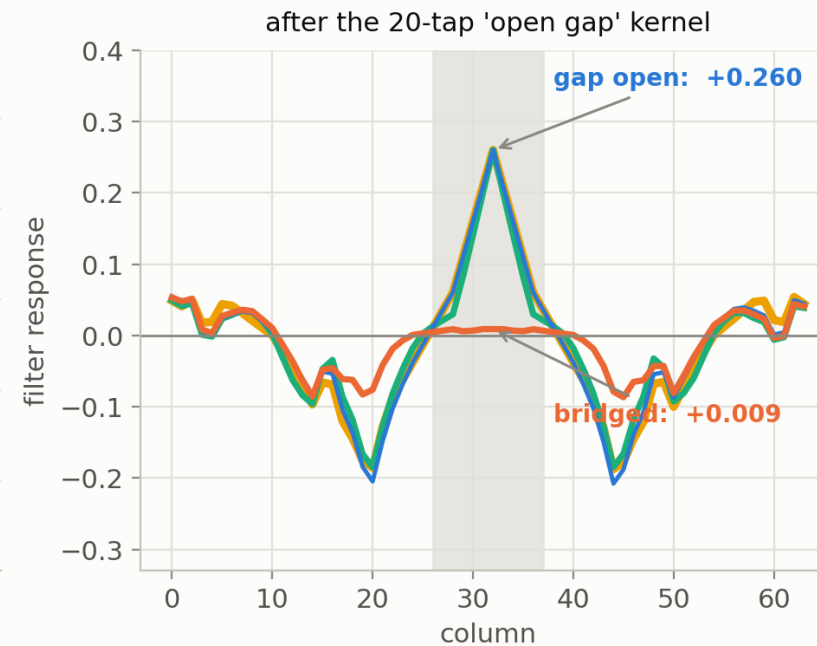
- Every crop is $64 \times 64 \times 3$ and centred on one two-pad footprint, but registration, placement tolerance and magnification mean the joint is never twice at the same address.
- We labelled 480 images from one afternoon on one line; the model will then be handed every shift, every lamp and every board.



Keep the two halves in mind: the classes are identical, the nuisance variation is not. That gap decides the whole lecture.

A kernel is a matched filter

- Row 38 of the crop runs through both fillets and the gap between them. A 20-tap kernel — positive on the shoulders, negative across the gap — is all a bridge detector needs.

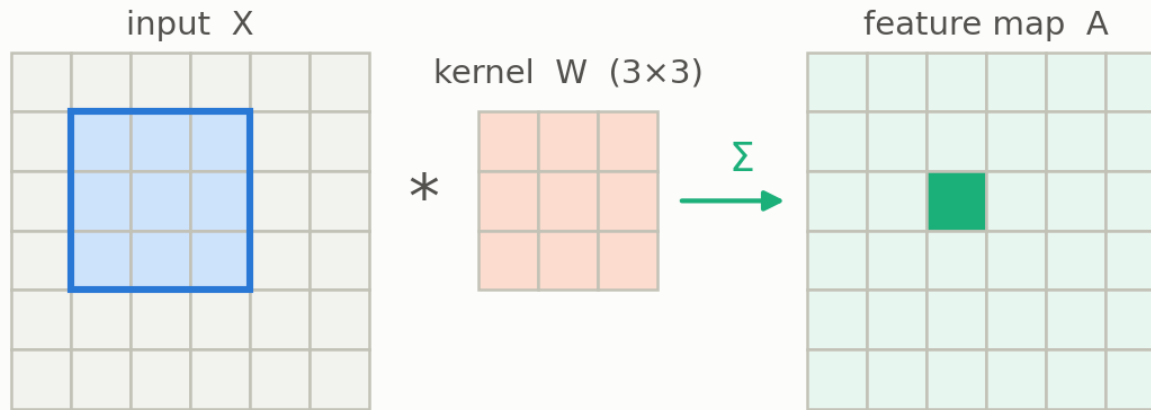


```
w = np.array([1.]*4 + [-1.]*12 + [1.]*4)    # shoulders, gap, shoulders
w = w - w.mean()    # zero-sum: a flat region of ANY brightness gives 0
y = np.convolve(x, w[::-1], mode='same')    # [::-1] -> cross-correlation
```

The same idea with two indices

- Padding and stride extend to both axes independently, so the output size is the one-dimensional formula applied per axis — predict it before you run, then check with a dummy batch.

one output pixel = one dot product with a local patch



output size

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

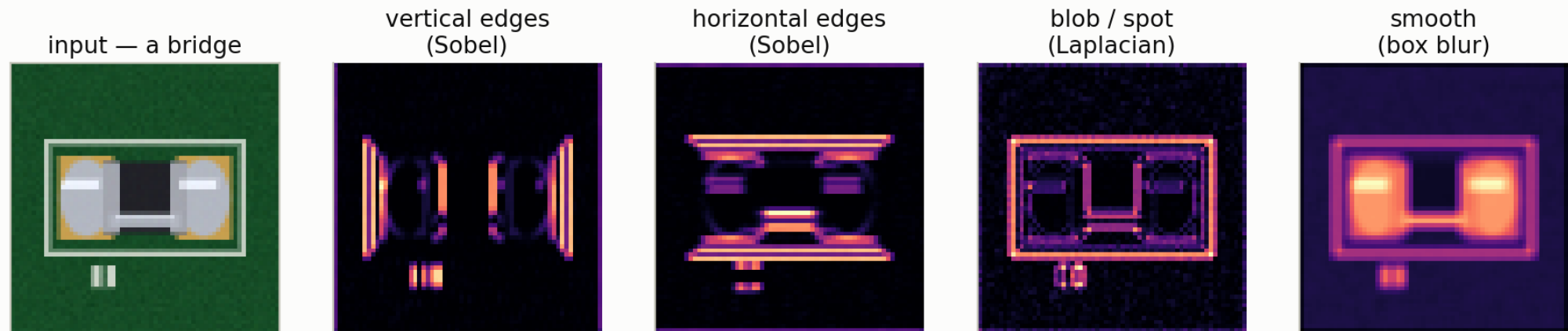
n	m	p	s	o	
64	3	1	1	64	same padding
64	3	0	1	62	valid – shrinks
64	2	0	2	32	the 2x2 pool

sparse connectivity: an output pixel sees only a 3x3 patch. parameter sharing: the same 9 weights slide over the whole image.

Same padding on the convolutions, and downsample deliberately with pooling or a stride-2 conv. Then you always know the shape, and you choose where the resolution is spent.

Nine numbers make a feature detector

- The classical kernels are not special: they are simply the weight patterns that respond to one local structure. A CNN learns these rather than being told them.



nine numbers per kernel — a CNN learns these instead of being told them

```
k = torch.tensor([[-1,0,1],[-2,0,2],[-1,0,1]], dtype=torch.float32)
out = F.conv2d(grey, k[None, None], padding=1) # cross-correlation!
# sum(k) == 0 -> blind to brightness, sensitive to structure
```

The three edge and blob kernels sum to zero and survive a lamp change; the box blur sums to one and destroys exactly the structure we need.

Pooling buys invariance, for no parameters

- Max-pooling reports the strongest response anywhere in a window, mean-pooling the average. Neither has a single trainable weight.

a 4×4 feature map, 2×2 windows

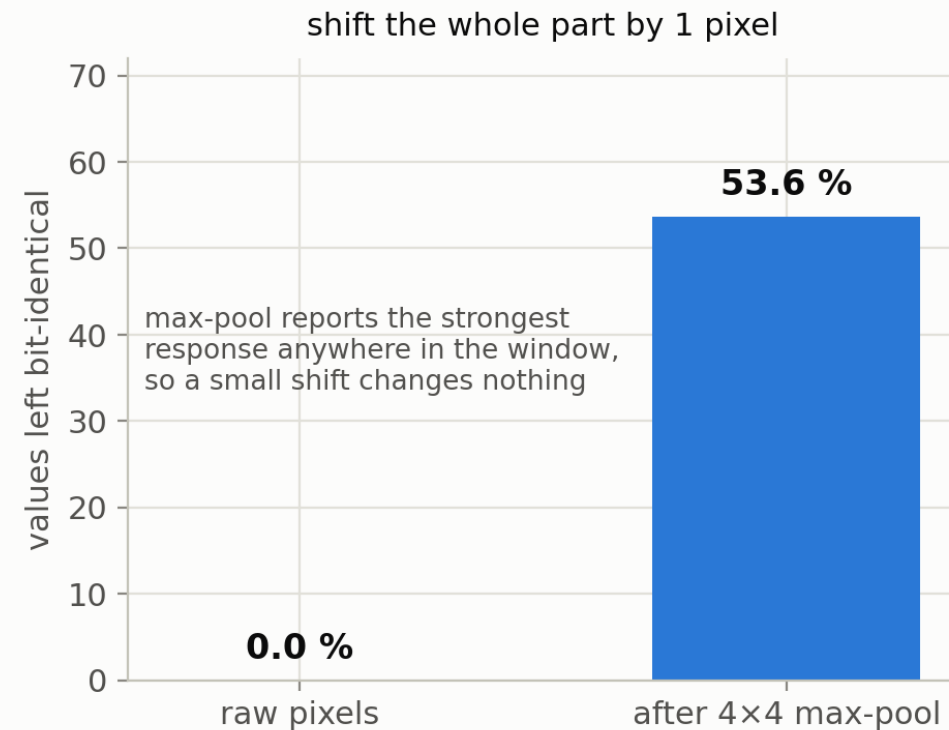
2	3	7	1
5	7	2	0
2	6	5	1
4	6	4	6

max-pool

7	7
6	6

mean-pool

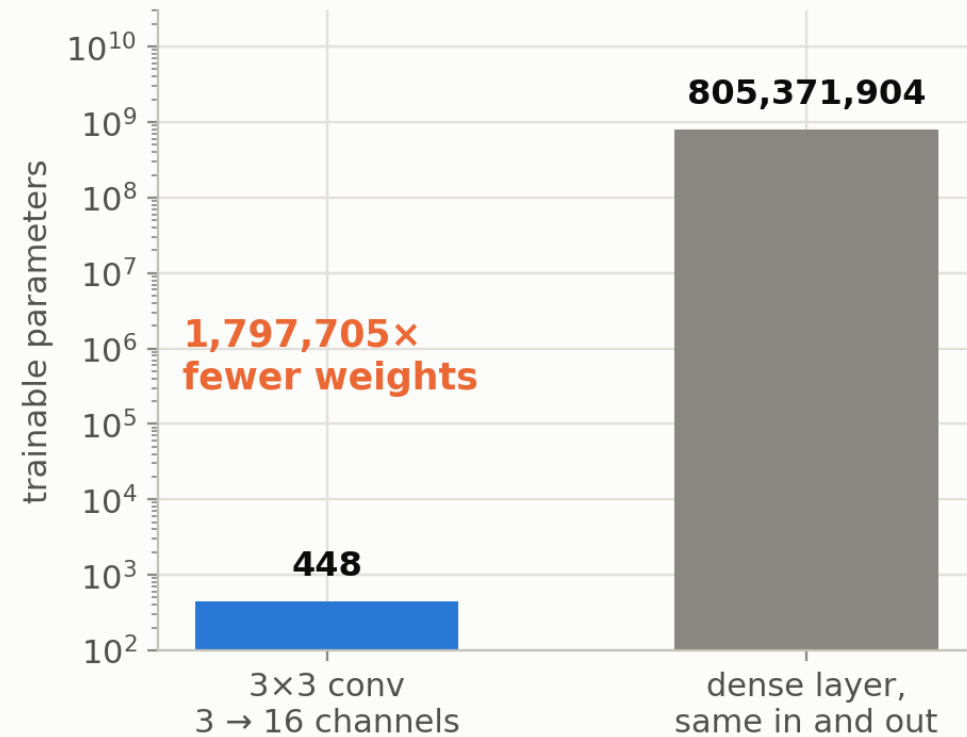
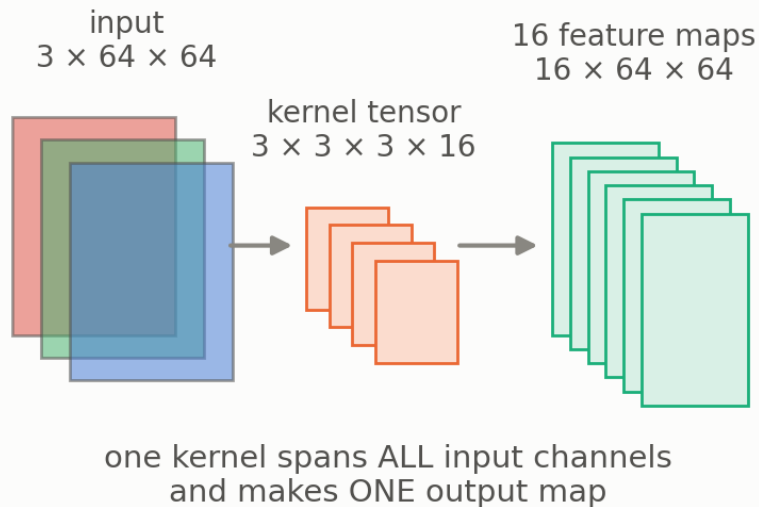
4.2	2.5
4.5	4.0



Move the whole part by one pixel: 0.0 % of raw pixels survive unchanged, but 53.6 % of the max-pooled values are bit-identical.

Channels, and the parameter argument

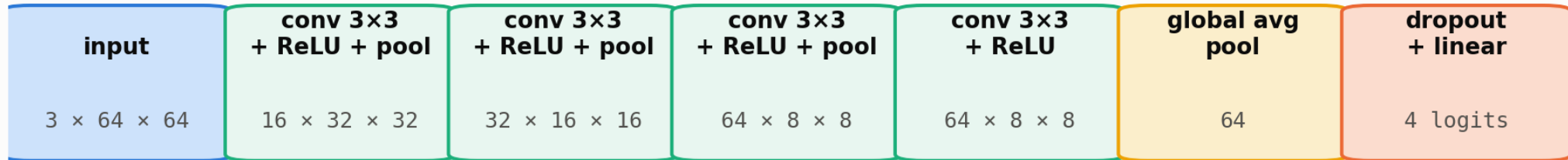
- A conv layer holds a rank-4 kernel tensor: one kernel per output channel, each spanning every input channel. PyTorch stores images as $N \times C \times H \times W$.



448 weights against 805,371,904 for a dense layer doing the same job — and the conv layer also encodes "a bridge looks the same wherever it appears", which is true.

Putting it together in torch.nn

- Four convolution blocks, channels doubling as the spatial size halves, then global average pooling and one linear layer — 60,772 parameters in total.



spatial size halves at every pool, channel count doubles — 60 772 parameters in total

```
model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 32x32
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 16x16
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 8x8
    nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(), # 64x8x8 -> 64
    nn.Dropout(0.3), nn.Linear(64, 4)) # raw logits, no Softmax
```

Choosing the loss, and the one bug everybody hits

- The task and the output format decide the loss. Prefer the versions that take raw logits: they fuse the log and the exponential, so nothing overflows when the model becomes confident.

```
# WRONG -- softmax applied twice; gradients squash, nothing errors
model = nn.Sequential(nn.Linear(64, 4), nn.Softmax(dim=1))
loss_fn = nn.CrossEntropyLoss()

# RIGHT -- the loss applies log-softmax itself, on raw logits
model = nn.Sequential(nn.Linear(64, 4))
loss_fn = nn.CrossEntropyLoss()      # targets are ints: y = 2, not one-hot
```

binary — one output unit

`nn.BCELoss` takes probabilities, needs a Sigmoid layer
`nn.BCEWithLogitsLoss` takes logits, no output activation

Both give 0.3711 on the notebook's example — until the logit is large, when only the fused version survives.

multi-class — one unit per class

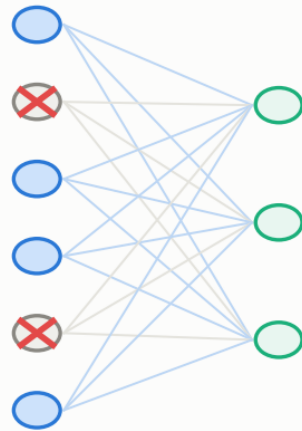
`nn.NLLLoss` takes log-probabilities, needs `LogSoftmax`
`nn.CrossEntropyLoss` takes logits, no output activation

Both give 0.6482 on the notebook's four-class example. Our four defect classes are mutually exclusive, so this is the right family.

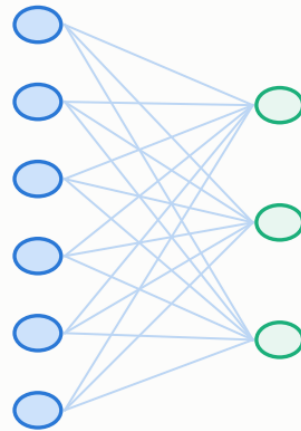
Dropout and L2 — and what they cannot fix

- Dropout zeroes each unit with probability p during training only, so the network cannot rely on any one unit. PyTorch rescales the survivors, so you must call `model.train()` and `model.eval()`.

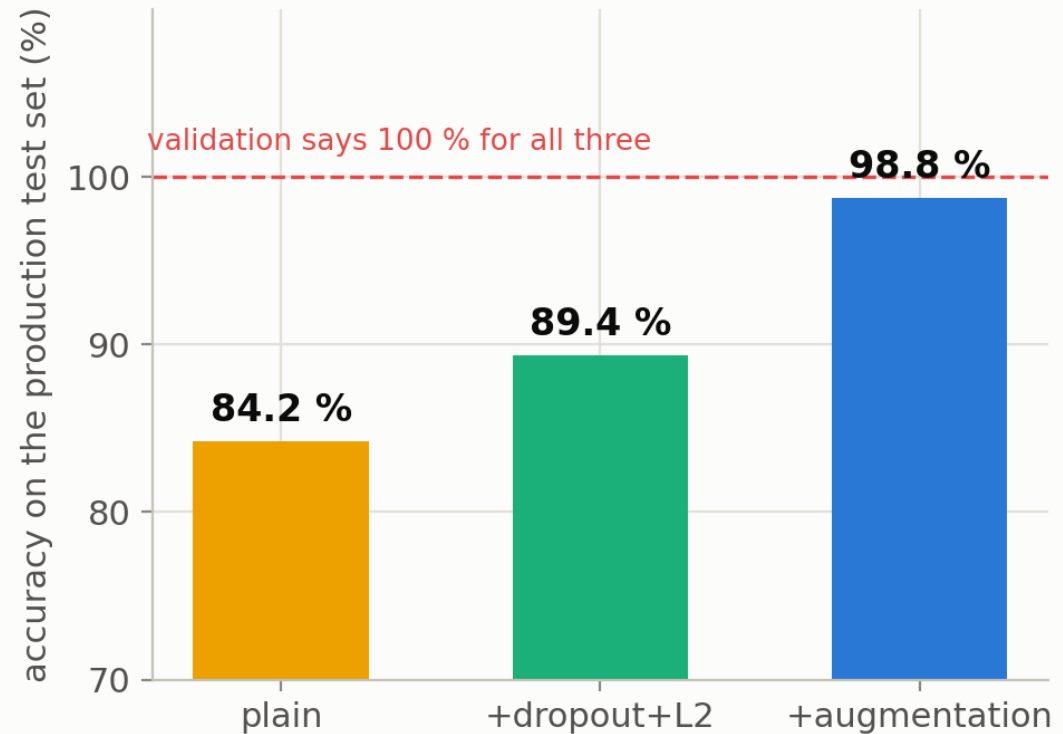
training ($p = 0.5$)



inference (all units)



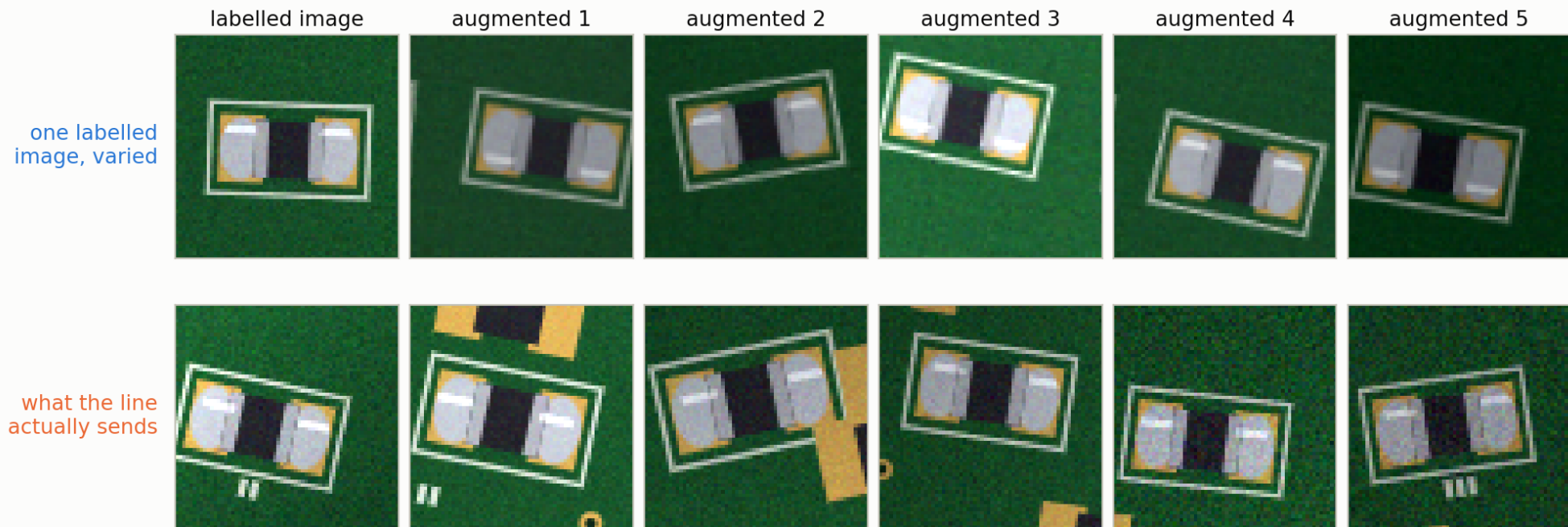
a different sub-network every mini-batch:
an ensemble you train for the price of one



Here they buy 5.1 points on production data — real, but far from enough. Dropout reduces variance from too many parameters; it cannot manufacture data you never collected.

Augmentation manufactures the missing data

- Random flips, $\pm 15^\circ$ rotation, ± 9 px shift and $\pm 30\%$ gain — each one a symmetry the inspection process really has, applied to the training images only.



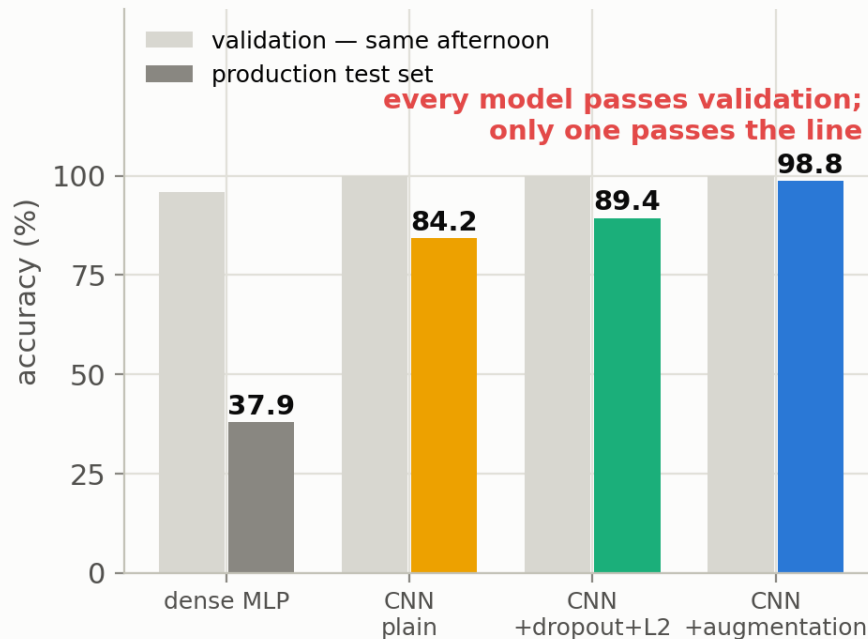
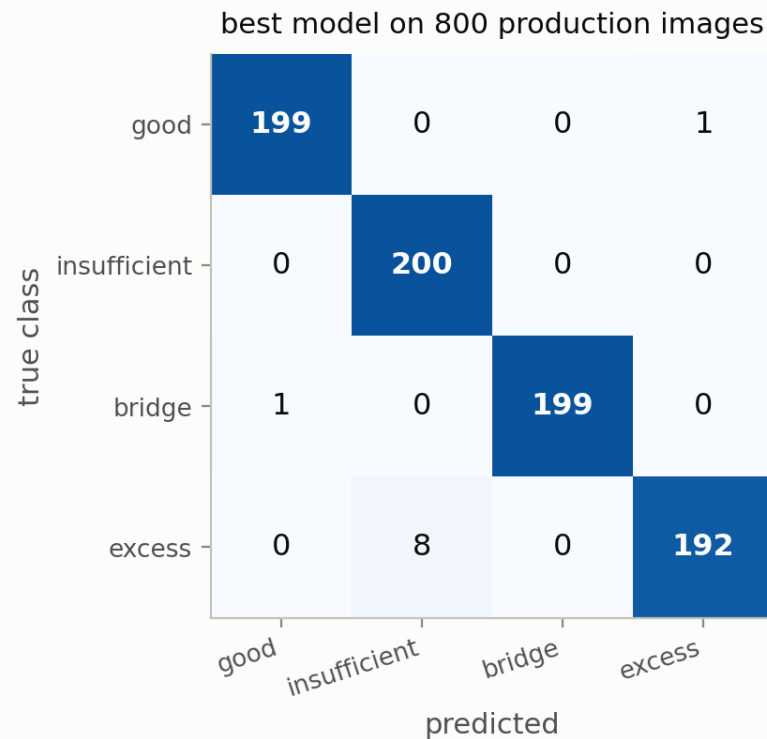
augmentation manufactures the variation the labelled set never saw — flips, $\pm 15^\circ$ rotation, ± 9 px shift, $\pm 30\%$ gain

Worth 14.5 points on production data — three times what dropout and L2 bought, because it attacks the actual problem: the labelled set never saw production.

RESULTS

Validation passed everything. The line did not.

- On the production test set: dense MLP 37.9 %, plain CNN 84.2 %, with regularisation 89.4 %, with augmentation 98.8 % — and the CNN has 52× fewer parameters than the MLP.

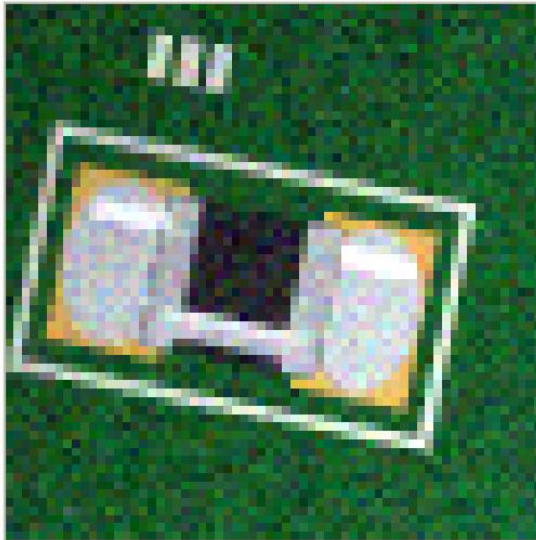


A validation split drawn from the same batch as your training data measures optimisation, not generalisation. If the model will be deployed, hold out data that looks like deployment.

Nobody specified these kernels

- The sixteen $3 \times 3 \times 3$ kernels of the first layer, plotted as RGB images, and the feature maps they produce for a real production frame.

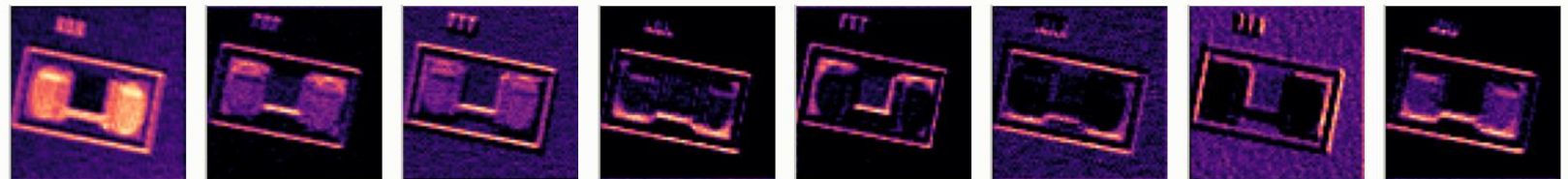
a bridge, as the line sent it



the 16 learned 3×3 kernels of conv1 (8 shown)



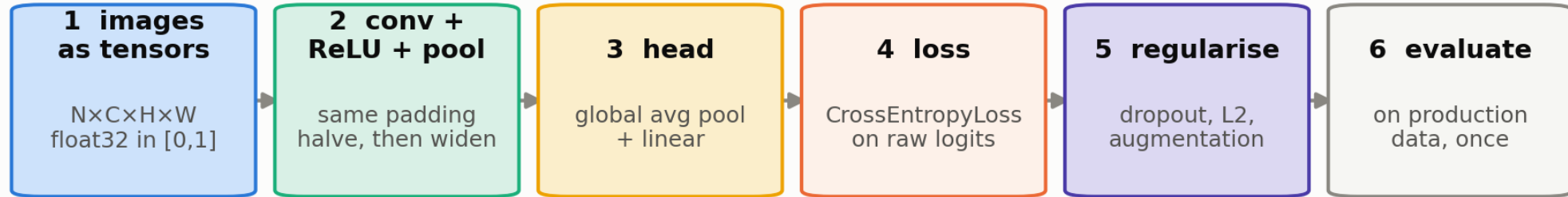
the feature maps they produce



oriented edge and colour-contrast detectors — nobody specified them

Early layers find edges and colour contrast; later layers combine them into "fillet present" and "gap bridged". That is the feature hierarchy, and it was learned from 480 labelled images.

What to take away



every image model you build follows this shape

1. A convolution is a dot product with a local patch, slid over the image. Nine numbers make a feature detector.
2. Predict every shape with $o = \lfloor (n + 2p - m)/s \rfloor + 1$, then check it with a dummy batch. Most CNN bugs are shape bugs.
3. Sparse connectivity and parameter sharing save parameters — and, more importantly, encode an assumption that is true.
4. Pooling costs no parameters and buys local translation invariance: 53.6 % of the representation survives a 1-pixel shift.
5. Never put a Softmax before CrossEntropyLoss, and always switch `model.train()` / `model.eval()` around dropout.
6. Dropout fixes too much model. Augmentation fixes too narrow a dataset. Diagnose before you reach for either.
7. Validate on data that looks like deployment — otherwise every model passes and you learn nothing.

Generated by Claude
and customized by



dew.ninja 2026